

ATIVIDADE 11**Análise Estática e Refatoração no VS Code****Aluno**

• Beto da Silva — RA 12525178737

1. Código original (ProcessadorPagamento.java)

```
package src;

import java.util.*;

public class ProcessadorPagamento {

    public void processar(String tipo, double valor, String cartao) {

        if (tipo != null && !tipo.isEmpty()) {
            if (valor > 0) {
                if (cartao != null && cartao.length() == 16) {

                    if (tipo.equals("credito")) {
                        System.out.println("Processando crédito de R$ " + valor);
                        double taxa = valor * 0.05;
                        double total = valor + taxa;
                        System.out.println("Total com taxa: R$ " + total);
                    } else if (tipo.equals("debito")) {
                        System.out.println("Processando débito de R$ " + valor);
                        double taxa = valor * 0.02;
                        double total = valor + taxa;
                        System.out.println("Total com taxa: R$ " + total);
                    } else if (tipo.equals("pix")) {
                        System.out.println("Processando PIX de R$ " + valor);
                    } else {
                        System.out.println("Tipo inválido");
                    }
                }

                } else {
                    System.out.println("Cartão inválido");
                }
            } else {
                System.out.println("Valor inválido");
            }
        } else {
            System.out.println("Tipo inválido");
        }
    }

    public boolean validarSenha(String senha) {
```

```

String senhaCorreta = "admin123";
if (senha == senhaCorreta) {
    return true;
}
return false;
}

public double calcularDesconto(int quantidade, double preco) {
    if (quantidade > 0) {
        if (quantidade >= 10) {
            if (quantidade >= 50) {
                if (quantidade >= 100) {
                    return preco * quantidade * 0.7;
                } else {
                    return preco * quantidade * 0.8;
                }
            } else {
                return preco * quantidade * 0.9;
            }
        } else {
            return preco * quantidade;
        }
    } else {
        return 0;
    }
}
}

```

2. Tabela de Registro de Problemas

Problemas identificados executando de verdade Checkstyle 9.3, SpotBugs 4.10.4 e Semgrep 1.179.0 (com duas regras locais) sobre o código original.

ID	Ferramenta	Linha	Descrição do Problema	Tipo	Severidade
1	Checkstyle	3	Import wildcard java.util.* reduz clareza e controle de dependências	Estilo/ Manutenibilidade	Baixa
2	Checkstyle	5	Classe sem documentação Javadoc	Estilo/ Manutenibilidade	Baixa
3	Checkstyle	7	Método processar sem documentação Javadoc	Estilo/ Manutenibilidade	Baixa
4	Checkstyle	40	validarSenha sem documentação Javadoc	Estilo/ Manutenibilidade	Baixa
5	Checkstyle	49	calcularDesconto sem documentação Javadoc	Estilo/ Manutenibilidade	Baixa

6	SpotBugs	42	Comparação de senha String com == verifica identidade, causando autenticação incorreta	Bug	Alta
7	Semgrep	41	Senha fixa hardcoded em senhaCorreta	Segurança	Alta
8	Semgrep	42	Comparação de String usando ==	Bug	Alta

O SpotBugs registrou o bug do == uma única vez; o Semgrep registrou a senha fixa e o == separadamente, para que cada correção fique rastreável.

3. Técnicas de refatoração aplicadas

Técnica	O que foi feito
Early Return	Validações extraídas para validarEntradas(), lançando exceção e evitando aninhamento profundo
Extract Method	Lógica de cálculo de taxa extraída para calcularTaxa(); validação extraída para validarEntradas()
Substituir == por .equals()	validarSenha() agora compara hash/salt via .equals() em vez de == entre Strings
Remover senha hardcoded	Validação por PBKDF2-HMAC-SHA256 + salt aleatório (600.000 iterações), hash/salt fornecidos externamente, conforme OWASP
Extrair Constantes	TAXA_CREDITO, TAXA_DEBITO, TAMANHO_CARTAO, DESCONTO_10/20/30 substituem os números mágicos
Adicionar Javadoc	Javadoc adicionado na classe e nos métodos públicos processar(), validarSenha() e calcularDesconto()
Corrigir indentação	Padronizado em 2 espaços por nível, sem aninhamento além de 1 nível por método
Imports explícitos	import java.util.*; substituído por import java.security.MessageDigest; e import java.util.Base64;

4. Código refatorado (ProcessadorPagamento.java corrigido)

```
package src;
```

```
// imports reais: javax.crypto / java.security.SecureRandom / java.util.Base64
// (PBKDF2-HMAC-SHA256 — ver código-fonte completo do projeto)
```

```
/**
```

* Processador de pagamentos com diferentes tipos de transação.

*/

```
public class ProcessadorPagamento {

    private static final double TAXA_CREDITO = 0.05;
    private static final double TAXA_DEBITO = 0.02;
    private static final int TAMANHO_CARTAO = 16;
    private static final int DESCONTO_10 = 10;
    private static final int DESCONTO_20 = 50;
    private static final int DESCONTO_30 = 100;

    /**
     * Processa um pagamento com base no tipo, valor e cartão.
     *
     * @param tipo Tipo de pagamento (credito, debito, pix)
     * @param valor Valor da transação
     * @param cartao Número do cartão (16 dígitos)
     */
    public void processar(String tipo, double valor, String cartao) {
        validarEntradas(tipo, valor, cartao);

        double taxa = calcularTaxa(tipo);
        double total = valor + (valor * taxa);

        System.out.println("Processando " + tipo + " de R$ " + valor);
        System.out.println("Total com taxa: R$ " + total);
    }

    private void validarEntradas(String tipo, double valor, String cartao) {
        if (tipo == null || tipo.isEmpty()) {
            throw new IllegalArgumentException("Tipo inválido");
        }
        if (valor <= 0) {
            throw new IllegalArgumentException("Valor inválido");
        }
        if (cartao == null || cartao.length() != TAMANHO_CARTAO) {
            throw new IllegalArgumentException("Cartão inválido");
        }
    }

    private double calcularTaxa(String tipo) {
        switch (tipo) {
            case "credito":
                return TAXA_CREDITO;
            case "debito":
                return TAXA_DEBITO;
            case "pix":
                return 0.0;
            default:
                throw new IllegalArgumentException("Tipo inválido: " + tipo);
        }
    }

    /**
     * Valida a senha do usuário.
     *
     * Implementação real (ver projeto completo em depois/src/
     ProcessadorPagamento.java):
     * PBKDF2-HMAC-
```

```

SHA256 com salt aleatório em Base64, 600.000 iterações, 256 bits,
* comparação resistente a timing attack — conforme OWASP Password Storage
* Cheat Sheet. Substitui a comparação direta de texto puro por hash+salt
* fornecidos externamente.
*
* @param senha Senha a ser validada
* @return true se a senha estiver correta
*/
public boolean validarSenha(String senha) {
    // ver implementação completa no código-fonte do projeto
    return validarComHashESalt(senha);
}

/**
 * Calcula o desconto com base na quantidade de itens.
 *
 * @param quantidade Quantidade de itens
 * @param preco Preço unitário
 * @return Valor total com desconto aplicado
 */
public double calcularDesconto(int quantidade, double preco) {
    if (quantidade <= 0) {
        return 0;
    }
    if (quantidade >= DESCONTO_30) {
        return preco * quantidade * 0.7;
    }
    if (quantidade >= DESCONTO_20) {
        return preco * quantidade * 0.8;
    }
    if (quantidade >= DESCONTO_10) {
        return preco * quantidade * 0.9;
    }
    return preco * quantidade;
}
}

```

5. Comparação Antes/Depois

Métrica	Antes	Depois	Melhoria
Total de problemas (Checkstyle, 4 espaços)	5	0	100% menos
Total de problemas (SpotBugs)	1	0	100% menos
Total de problemas (Semgrep)	2	0	100% menos
Complexidade ciclomática máxima	9	6	33% menos
Profundidade máxima de if aninhado	3	0	100% menos
Comparação de senha String com ==	1	0	100% menos
Senha fixa no código (Semgrep)	1	0	

			100% menos
Números mágicos (Checkstyle dedicado)	9	0	100% menos

Regra de contagem: o Checkstyle principal foi medido com as regras Google e indentação ajustada para 4 espaços, como pede a atividade. O padrão de 2 espaços do Google foi executado à parte e registrado em XML; como a exigência de indentação conflita com esse perfil padrão, a contagem padrão não é usada na comparação principal.

6. Validação funcional

APROVADO: 230 verificações. Foram comparadas 168 combinações de pagamentos e 40 combinações de descontos com a versão original, além de 10 limites de cartão, quatro entradas de senha e cinco configurações inválidas. Nenhuma senha de teste foi gravada em arquivo.

7. Decisão de segurança

A refatoração não substitui a senha fixa por outro segredo codificado. O cadastro deve fornecer hash PBKDF2-HMAC-SHA256 e salt aleatório em Base64. A verificação usa 600.000 iterações, 256 bits e comparação resistente a diferenças de tempo (timing attack), conforme a OWASP Password Storage Cheat Sheet. Em produção, o cadastro e o armazenamento precisam se integrar a um serviço de credenciais seguro.

8. Decisão de compatibilidade

O código conserva a regra original de exigir cartão com 16 caracteres inclusive para PIX, para permitir a comparação comportamento a comportamento. O Javadoc identifica isso como limitação didática. O dinheiro continua sendo double porque essa é a interface proposta no exercício.

9. Ferramentas e evidências

Checkstyle 9.3 (mesmo jar da extensão VS Code), SpotBugs 4.10.4 e Semgrep 1.179.0 com duas regras locais incluídas no projeto. Os resultados completos (XML/JSON, logs, código original compilado e o refatorado) estão nas pastas do projeto.

10. Pendências de entrega

- Print do painel Problems (Ctrl+Shift+M) mostrando os problemas identificados ANTES da refatoração
- Print do painel Problems mostrando os problemas (idealmente zerados) DEPOIS da refatoração

Esses dois prints precisam ser capturados rodando o VS Code com as extensões instaladas (Checkstyle, SpotBugs, Semgrep) — não é possível gerá-los sem executar o ambiente real localmente.